

Prototype vs instance properties



```
function Counter () {
  var count = 0;
  this.incrementCounter = function () {
    count++;
    console.log(count);
  }

  this.decrementCounter = function () {
    count--;
    console.log(count);
  }
}
```

```
var counter1 = new Counter();
counter1.count = 999;
counter1.incrementCounter();
console.log(counter1.count)
Counter.vipul = 50;
console.log(Counter.count)
console.log(Counter.vipul)
console.log(counter1.vipul)
```

in this when we added vipul to the Counter object property ior prototype of Counter than why it is not available to counter1 which is the instance of Counter.

Excellent question, Vipul 🙌 — and this is actually one of those core JavaScript concepts that reveals how **constructor functions**, **instances**, and **prototypes** really work.

Let's go step-by-step carefully 📌

♦ Step 1 — What Counter really is

Counter is **just a function** — but when you call it with **new Counter()**, it acts like a **constructor function** that creates a new object and sets up a relationship between the new object and **Counter.prototype**.

So:

```
js

function Counter() { ... }

var counter1 = new Counter();
```

Now:

- `Counter` → is a **function object**.
- `Counter.prototype` → is an object that will be the **prototype of instances** created by `new Counter()`.
- `counter1` → is a **new object**, linked to `Counter.prototype`.

♦ Step 2 — What happens when you write this:

```
js
Counter.vipul = 50;
```

You are adding a property **directly to the function object** itself — not to its prototype.

Think of this like:

```
js
// This is a static property
FunctionObject.PropertyName = value;
```

So now:

```
js
Counter.vipul → 50
```

But the **instances (like `counter1`) are not linked to the constructor function**, they are linked to its **prototype**.

So:

```
js
counter1.__proto__ === Counter.prototype ✓
counter1.__proto__ !== Counter ✗
```

That's why:

```
js
console.log(counter1.vipul); // undefined ✗
```

because `vipul` is on the `Counter` function, not on `Counter.prototype`.

♦ Step 3 — If you wanted `counter1` to access `vipul`

Then you should attach it to the **prototype**, like this:

```
js
Counter.prototype.vipul = 50;
```

Now every instance created with `new Counter()` will be able to access it:

```
js
var counter1 = new Counter();
console.log(counter1.vipul); //  50
```

Because prototype chain lookup works like this:

```
javascript
counter1 → Counter.prototype → Object.prototype → null
```

♦ Step 4 — Quick comparison

Property added to...	Syntax	Accessible by instances?	Example
Constructor function itself	<code>Counter.vipul = 50</code>	 No	<code>counter1.vipul</code> → undefined
Constructor's prototype	<code>Counter.prototype.vipul = 50</code>	 Yes	<code>counter1.vipul</code> → 50

Summary

- `Counter.vipul = 50` → adds a **static property** on the constructor function (not inherited by instances).
- `Counter.prototype.vipul = 50` → adds a **shared property** that all instances can access through prototype chain.

Would you like me to visualize this with a small **diagram** showing how the `Counter`, `Counter.prototype`, and `counter1` are linked in memory? It helps make this concept crystal clear.